Lane2 Language Specification

Version: v1 draft
Status: working specification

Contents

1 Introduction

Lane2 is a strict, pure, expression-oriented functional programming language. Its first implementation target is a parser, a type checker, and an AST interpreter; later implementations may add bytecode compilation, a virtual machine, a linker, and effect handling.

Lane2 is intentionally small. It has no mutable state, assignment, trait system, method dispatch, module system, or implicit typeclass-style instance search in v1. Instead, v1 is centered on nominal data, first-class functions, bidirectional local type inference, exhaustive pattern matching, and contextual resolution of explicitly offered values.

This document specifies Lane2/Core v1: the platform-independent part of Lane2 that should behave the same across the initial AST interpreter and later execution backends.

1.1 Scope

Lane2/Core v1 covers:

- source structure and declarations;
- lexical scope and name resolution;
- nominal struct and enum types;
- function and block expressions;
- local type inference;
- pattern matching;
- contextual resolution and operation-based operators;
- unsafe builtin expressions.

The following are outside Lane2/Core v1:

- mutation and assignment;
- modules and imports;
- bytecode and virtual machine semantics;
- linker entrypoint selection;
- algebraic effects;
- type aliases;
- traits, typeclasses, and interfaces;
- tuples and collection syntax.

1.2 Compatibility

This specification is a v1 draft. No compatibility is promised between draft revisions. Language rules may be added, removed, or changed while the first implementation is being developed.

Once a v1 implementation is declared stable, this section should be replaced by an explicit compatibility policy.

1.3 Experimental Features

This draft does not distinguish stable and experimental language features. Every rule in this document is provisional until v1 is stabilized.

Future drafts may mark individual features as experimental when their surface syntax or semantics are intentionally unsettled.

1.4 Feedback

Design feedback is tracked in the Lane2 repository. Implementation changes should update this specification, `CONTEXT.md`, and ADRs when the change affects user-visible semantics.

1.5 Reference

When referencing this draft, use:

> Lane2 Language Specification : Lane2/Core v1 draft.

2 Syntax and Grammar

2.1 Notation

```
grammar        ::= { production }
production     ::= nonTerminal "::=" grammarExpression
grammarExpression ::=
    grammarSequence { "|" grammarSequence }
grammarSequence ::=
    { grammarItem }
grammarItem ::=
    terminal
  | nonTerminal
  | "[" grammarExpression "]"
  | "{" grammarExpression "}"
  | "(" grammarExpression ")"
  | grammarItem "?"
  | grammarItem "*"
  | grammarItem "+"

terminal       ::= "'" terminalText "'"
nonTerminal    ::= lowerCamelIdentifier

empty sequence denotes epsilon.
"A B" denotes sequencing.
"A | B" denotes choice.
"[ A ]" and "A?" denote optional occurrence.
"{ A }" and "A*" denote zero or more occurrences.
"A+" denotes one or more occurrences.
Parenthesized grammar expressions group without producing syntax.
```

2.2 Lexical Grammar

```
lexicalInput ::=
    (trivia | token)* eof

trivia ::=
    whitespace
  | lineTerminator
  | comment

token ::=
    keyword
  | reservedWord
  | identifier
  | intLiteral
  | stringLiteral
  | boolLiteral
  | operatorToken
  | punctuationToken

keyword ::=
```

```
    "struct"
  | "enum"
  | "fn"
  | "let"
  | "auto"
  | "offer"
  | "if"
  | "else"
  | "match"
  | "builtin"

reservedWord ::=
    "effect"
  | "handler"
  | "module"
  | "import"
  | "pub"
  | "type"
  | "return"

identifier ::=
    asciiIdentifierStart identifierContinue*
  | "_" identifierContinue+

identifierContinue ::=
    "_"
  | asciiIdentifierStart
  | decimalDigit

boolLiteral ::=
    "true"
  | "false"

intLiteral ::=
    decimalDigit+

stringLiteral ::=
    "\"" stringElement* "\""

stringElement ::=
    asciiStringCharacter
  | stringEscape

asciiStringCharacter ::=
    asciiCodePointExceptControlBackslashQuote

stringEscape ::=
    "\\" "\\"
  | "\\" "\""
  | "\\" "n"
  | "\\" "r"
  | "\\" "t"
  | "\\" "x" asciiHexByte

asciiHexByte ::=
    asciiHexLowByte
  | asciiHexHighByte

asciiHexLowByte ::=
    ("0" | "1" | "2" | "3" | "4" | "5" | "6") asciiHexDigit
```

```
asciiHexHighByte ::=
    "7" ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7")

operatorToken ::=
    "+"
  | "-"
  | "*"
  | "/"
  | "%"
  | "=="
  | "!="
  | "<"
  | "<="
  | ">"
  | ">="
  | "&&"
  | "||"
  | "!"
  | "|>"

punctuationToken ::=
    "("
  | ")"
  | "{"
  | "}"
  | "["
  | "]"
  | ","
  | "."
  | ":"
  | "::"
  | "->"
  | "=>"
  | "="
  | "_"

whitespace ::=
    U+0009
  | U+000B
  | U+000C
  | U+0020

lineTerminator ::=
    U+000A
  | U+000D
  | U+000D U+000A

decimalDigit ::=
    "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

asciiHexDigit ::=
    decimalDigit
  | "a" | "b" | "c" | "d" | "e" | "f"
  | "A" | "B" | "C" | "D" | "E" | "F"

asciiIdentifierStart ::=
    asciiUppercaseLetter
  | asciiLowercaseLetter

asciiUppercaseLetter ::=
    any ASCII code point from U+0041 through U+005A
```

```
asciiLowercaseLetter ::=
    any ASCII code point from U+0061 through U+007A
```

Lexical analysis uses maximal munch. If more than one token class matches the same maximal source span, keyword, reservedWord, and boolLiteral take priority over identifier.

2.3 Syntax Grammar

```
sourceFile ::=
    topLevelDeclaration* eof

topLevelDeclaration ::=
    structDeclaration
  | enumDeclaration
  | functionDeclaration
  | topLevelLetDeclaration
  | topLevelOfferedLetDeclaration
  | offerDeclaration

structDeclaration ::=
    "struct" typeName typeParameters? "{" structMember* "}"

structMember ::=
    structFieldDeclaration

structFieldDeclaration ::=
    fieldModifier? fieldName space ":" space type

fieldModifier ::=
    "offer"

enumDeclaration ::=
    "enum" typeName typeParameters? "{" enumVariant* "}"

enumVariant ::=
    variantName enumPayload?

enumPayload ::=
    "(" commaSeparatedTypes? ")"

functionDeclaration ::=
    "fn" typeParameters? functionName "(" commaSeparatedParameters? ")" "->" type block

parameter ::=
    parameterModifiers? valueName space ":" space type

parameterModifiers ::=
    parameterModifier+

parameterModifier ::=
    "auto"
  | "offer"

topLevelLetDeclaration ::=
    "let" valueName space ":" space type "=" expression

topLevelOfferedLetDeclaration ::=
    "let" "offer" valueName space ":" space type "=" expression

localLetDeclaration ::=
```

```
    "let" valueName typeAnnotation? "=" expression

localOfferedLetDeclaration ::=
    "let" "offer" valueName typeAnnotation? "=" expression

typeAnnotation ::=
    space ":" space type

offerDeclaration ::=
    "offer" valueName

type ::=
    typeConstructor typeArguments?
  | functionType

typeConstructor ::=
    typeName

typeArguments ::=
    "[" commaSeparatedTypes "]"

typeParameters ::=
    "[" commaSeparatedTypeParameters "]"

functionType ::=
    typeParameters? "(" commaSeparatedTypes? ")" "->" type

expression ::=
    ifExpression
  | matchExpression
  | functionLiteral
  | block
  | pipelineExpression

pipelineExpression ::=
    binaryExpression { "|>" pipelineRhs }

pipelineRhs ::=
    callExpression
  | functionLiteral

binaryExpression ::=
    unaryExpression { binaryOperator unaryExpression }

unaryExpression ::=
    unaryOperator unaryExpression
  | callExpression

callExpression ::=
    fieldExpression { callSuffix }

callSuffix ::=
    "(" commaSeparatedCallArguments? ")"

callArgument ::=
    expression
  | valueName "=" expression

fieldExpression ::=
    primaryExpression { "." fieldName }
```

```
primaryExpression ::=
    literal
  | valueName
  | qualifiedVariantExpression
  | structLiteral
  | builtinExpression
  | "(" expression ")"
  | "(" ")"

literal ::=
    intLiteral
  | stringLiteral
  | boolLiteral

qualifiedVariantExpression ::=
    typeName typeArguments? "::" variantName variantArguments?

variantArguments ::=
    "(" commaSeparatedExpressions? ")"

structLiteral ::=
    typeName typeArguments? "::" "{" commaSeparatedStructLiteralFields "}"

structLiteralField ::=
    fieldName
  | fieldName ":" expression

builtinExpression ::=
    "builtin" "(" stringLiteral ")"

functionLiteral ::=
    "fn" typeParameters? "(" commaSeparatedFunctionLiteralParameters? ")"
functionReturnAnnotation? block

functionLiteralParameter ::=
    functionLiteralParameterModifiers? valueName
  | functionLiteralParameterModifiers? valueName space ":" space type

functionLiteralParameterModifiers ::=
    "offer"+

functionReturnAnnotation ::=
    "->" type

block ::=
    "{" localItem* expression "}"

localItem ::=
    localLetDeclaration
  | localOfferedLetDeclaration
  | functionDeclaration
  | offerDeclaration

ifExpression ::=
    "if" expression block "else" block

matchExpression ::=
    "match" expression "{" matchArm* "}"

matchArm ::=
    pattern "=>" expression
```

```
pattern ::=
    "_"
  | valueName
  | literal
  | qualifiedVariantPattern
  | structPattern

qualifiedVariantPattern ::=
    typeName "::" variantName patternArguments?

patternArguments ::=
    "(" commaSeparatedPatterns? ")"

structPattern ::=
    typeName "::" "{" commaSeparatedStructPatternFields "}"

structPatternField ::=
    fieldName
  | fieldName ":" pattern

binaryOperator ::=
    "+" | "-" | "*" | "/" | "%"
  | "==" | "!=" | "<" | "<=" | ">" | ">="
  | "&&" | "||"

unaryOperator ::=
    "-" | "!"

commaSeparatedTypes ::=
    type ("," type)* ","?

commaSeparatedTypeParameters ::=
    typeParameter ("," typeParameter)* ","?

commaSeparatedParameters ::=
    parameter ("," parameter)* ","?

commaSeparatedExpressions ::=
    expression ("," expression)* ","?

commaSeparatedCallArguments ::=
    callArgument ("," callArgument)* ","?

commaSeparatedFunctionLiteralParameters ::=
    functionLiteralParameter ("," functionLiteralParameter)* ","?

commaSeparatedStructLiteralFields ::=
    structLiteralField ("," structLiteralField)* ","?

commaSeparatedPatterns ::=
    pattern ("," pattern)* ","?

commaSeparatedStructPatternFields ::=
    structPatternField ("," structPatternField)* ","?

typeName ::=
    identifier

functionName ::=
    identifier
```

```
valueName ::=
    identifier

fieldName ::=
    identifier

variantName ::=
    identifier

typeParameter ::=
    identifier

space ::=
    whitespace+
```

The precedence and associativity of `binaryOperator`, `unaryOperator`, calls, field access, and pipeline expressions are defined in “Operators”.

2.4 Comments

```
comment ::=
    lineComment
  | blockComment

lineComment ::=
    "/" "/" lineCommentCharacter* lineTerminator?

lineCommentCharacter ::=
    any Unicode code point except U+000A or U+000D

blockComment ::=
    "/" "*" blockCommentCharacter* "*" "/"

blockCommentCharacter ::=
    any Unicode code point sequence that does not start "*/"
```

Comments are trivia. Comments do not appear in the syntactic grammar.

3 Type System

3.1 Notations

Notation	Meaning
T, U, R, F, P, Q	types
A	type variable
C	type constructor
S	struct type constructor
E	enum type constructor
e, c, f, b	expressions
a	match arm
x	value binder
i, j, k, n, m, q	indices and natural numbers
l	struct field name
v	enum variant name
Δ	type-constructor context
Θ	type-variable context
Γ	value typing context
D	custom type definition
$C \to D \in \Delta$	visible custom type definition binding
$\text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m)$	struct definition
$\text{enum}\left(A_1, ..., A_n; v_{j\left(P_{j,1}, ..., P_{j,m_j}\right)}\right)$	enum definition
$\text{params}(D) = (A_1, ..., A_n)$	custom type parameters
$D \vdash v : (P_1, ..., P_m)$	variant payload sequence declared by a custom type definition
$D \vdash \text{variants}\left(v_1, ..., v_q\right)$	variant sequence declared by a custom type definition
$\Delta\Theta \vdash E[T_1, ..., T_n] :: v \to (Q_1, ..., Q_m)$	instantiated variant payload sequence
σ	type substitution environment
$\sigma = [T_1/A_1, ..., T_n/A_n]$	substitution environment binding
$U[T_1/A_1, ..., T_n/A_n]$	direct type substitution
$U\sigma$	type substitution by environment
$\Delta\Theta \vdash T \;\; \text{type}$	well-formed type
$C[T_1, ..., T_n]$	nominal type application
$\forall A_1, ..., A_n.T$	universal type
$T \equiv U$	type equality
$\Gamma \vdash e : T$	expression typing
$\text{fields}(S[T_1, ..., T_n]) = (l_1 : Q_1, ..., l_m : Q_m)$	instantiated struct fields
$\text{arm-type}(E[T_1, ..., T_n], v, R)$	typed enum match arm
$\frac{P}{Q}$	rule with premise P and conclusion Q
name	abstract syntax constructor

Table 1: Type system notations

3.2 Primitive Types

Lane2/Core v1 has four primitive types: `Unit`, `Bool`, `Int`, and `String`.

Primitive formation. Each primitive type is well-formed in every type environment.

$$\Delta\Theta \vdash \text{Unit} \;\; \text{type}$$

$$\Delta\Theta \vdash \text{Bool} \;\; \text{type}$$

$$\Delta\Theta \vdash \text{Int} \;\; \text{type}$$

$$\Delta\Theta \vdash \text{String} \;\; \text{type}$$

Unit introduction. The expression `()` introduces a `Unit` value.

```
() // unit literal
```

$$\Gamma \vdash () : \text{Unit}$$

Unit elimination. Lane2/Core v1 has no dedicated elimination form for `Unit`.

Bool introduction. Boolean literals introduce `Bool` values.

```
true  // boolean literal
false // boolean literal
```

$$\Gamma \vdash \text{true} : \text{Bool}$$

$$\Gamma \vdash \text{false} : \text{Bool}$$

Bool elimination. `if` eliminates a `Bool` value.

```
if c {
  e1
} else {
  e2
} // c : Bool, e1 : T, e2 : T
```

$$\frac{\Gamma \vdash c : \text{Bool} \quad \Gamma \vdash e_1 : T \quad \Gamma \vdash e_2 : T}{\Gamma \vdash \text{if}(c, e_1, e_2) : T}$$

Int introduction. Integer literals introduce `Int` values.

```
n // integer literal
```

$$\Gamma \vdash n : \text{Int}$$

Int elimination. Lane2/Core v1 has no built-in syntactic eliminator for `Int`. Integer operations are typed through required intrinsics and prelude operation values.

String introduction. ASCII string literals introduce `String` values.

```
"..." // ASCII string literal
```

$$\Gamma \vdash s : \text{String}$$

String elimination. Lane2/Core v1 has no built-in syntactic eliminator for `String`. String equality is typed through the prelude.

Primitive type equality. Primitive types are equal only to themselves.

$$\text{Unit} \equiv \text{Unit}$$

$$\text{Bool} \equiv \text{Bool}$$

$$\text{Int} \equiv \text{Int}$$

$$\text{String} \equiv \text{String}$$

3.3 Function Types

Function formation.

```
fn f(x1 : T1, ..., xn : Tn) -> R { e } // function definition
let f : (T1, ..., Tn) -> R = fn(x1, ..., xn) { e } // function literal binding
```

$$\frac{\Delta\Theta \vdash T_1 \ \text{type} \quad \ldots \quad \Delta\Theta \vdash T_n \ \text{type} \quad \Delta\Theta \vdash R \ \text{type}}{\Delta\Theta \vdash (T_1, \ldots, T_n) \to R \ \text{type}}$$

Function introduction.

```
fn(x1 : T1, ..., xn : Tn) -> R { e } // function literal
fn f(x1 : T1, ..., xn : Tn) -> R { e } // function definition
```

$$\frac{\Gamma, x_1 : T_1, \ldots, x_n : T_n \vdash e : R}{\Gamma \vdash \text{fn}((x_1 : T_1), \ldots, (x_n : T_n), R, e) : (T_1, \ldots, T_n) \to R}$$

Function elimination.

```
f(a1, ..., an) // function call
```

$$\frac{\Gamma \vdash f : (T_1, \ldots, T_n) \to R \quad \Gamma \vdash a_1 : T_1 \quad \ldots \quad \Gamma \vdash a_n : T_n}{\Gamma \vdash f(a_1, \ldots, a_n) : R}$$

Function type equality.

$$\frac{T_1 \equiv U_1 \quad \ldots \quad T_n \equiv U_n \quad R \equiv S}{(T_1, \ldots, T_n) \to R \equiv (U_1, \ldots, U_n) \to S}$$

Function types are uncurried. `(T1, T2) -> R` is not the same type object as `(T1) -> (T2) -> R`.

3.4 Generic Function Types

Generic function formation.

```
[A1, ..., An](T1, ..., Tm) -> R // generic function type
```

$$\frac{\Delta\Theta, A_1, ..., A_n \vdash (T_1, ..., T_m) \to R \ \ \text{type}}{\Delta\Theta \vdash \forall A_1, ..., A_n.(T_1, ..., T_m) \to R \ \ \text{type}}$$

Generic function introduction.

```
fn[A1, ..., An](x1 : T1, ..., xm : Tm) -> R { e } // generic function literal
fn[A1, ..., An] f(x1 : T1, ..., xm : Tm) -> R { e } // generic function definition
```

A generic function literal or named generic function introduces a generic function value. The function body is checked under the declared type parameters and value parameters.

$$\frac{\Delta\Theta, A_1, ..., A_n \vdash (T_1, ..., T_m) \to R \ \ \text{type} \quad \Gamma, x_1 : T_1, ..., x_m : T_m \vdash e : R}{\Gamma \vdash \text{fn}([A_1, ..., A_n], (x_1 : T_1), ..., (x_m : T_m), R, e) : \forall A_1, ..., A_n.(T_1, ..., T_m) \to R}$$

Generic function elimination.

Generic function calls instantiate type parameters at the use site when the use site is unambiguous.

```
f(a1, ..., am) // generic function call with inferred type arguments
```

$$\frac{\Gamma \vdash f : \forall A_1, ..., A_n.(T_1, ..., T_m) \to R \quad \sigma = [U_1/A_1, ..., U_n/A_n] \quad \Gamma \vdash a_1 : T_1\sigma \quad ... \quad \Gamma \vdash a_m : T_m\sigma}{\Gamma \vdash f(a_1, ..., a_m) : R\sigma}$$

The substitution σ must be the unique substitution selected by local type inference for the call site. If no unique substitution exists, the call is ill-typed.

Generic function type equality.

$$\frac{(T_1, ..., T_m) \to R \equiv (U_1, ..., U_m) \to S}{\forall A_1, ..., A_n.(T_1, ..., T_m) \to R \equiv \forall A_1, ..., A_n.(U_1, ..., U_m) \to S}$$

Generic function type equality compares the number of type parameters and the function types under corresponding bound type parameters.

3.5 Nominal Type Constructors

Struct and enum declarations introduce nominal custom type definitions in Δ. This section specifies the shared treatment of nominal type constructors. Struct-specific and enum-specific declaration rules are specified in "Struct Types" and "Enum Types".

Top-level custom type declarations are checked in two phases. First, all top-level struct and enum constructors are collected into Δ as declaration-shape bindings. Second, every field type and variant payload type is checked under the collected Δ and the declaration's type parameters. This permits mutually recursive custom types.

Custom type collection. A top-level custom type declaration contributes its nominal constructor and declaration shape to Δ before member type checking begins. In enum declaration shapes, P_j denotes the payload type sequence of variant v_j.

```
struct S[A1, ..., An] { l1 : F1, ..., lm : Fm }
enum E[A1, ..., An] { vj(Pj1, ..., Pjmj) }
```

$$S \to \text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m) \in \Delta \quad E \to \text{enum}\big(A_1, ..., A_n; v_1(P_1), ..., v_{q(P_q)}\big) \in \Delta$$

Nominal formation. A nominal type application is well-formed when its custom type constructor is visible, the number of type arguments matches the declaration's type parameters, and every type argument is well-formed.

$$\frac{C \to D \in \Delta \quad \text{params}(D) = (A_1, ..., A_n) \quad \Delta\Theta \vdash T_1 \ \ \text{type} \quad ... \quad \Delta\Theta \vdash T_n \ \ \text{type}}{\Delta\Theta \vdash C[T_1, ..., T_n] \ \ \text{type}}$$

Nominal type equality. Nominal type equality compares constructor identity and then compares type arguments pairwise. There is no equality rule whose conclusion relates different nominal constructors.

$$\frac{T_1 \equiv U_1 \quad ... \quad T_n \equiv U_n}{C[T_1, ..., T_n] \equiv C[U_1, ..., U_n]}$$

3.6 Struct Types

Struct declaration well-formedness. A struct declaration is well-formed when every declared field type is well-formed under the collected type-constructor context and the struct's type parameters.

```
struct S[A1, ..., An] {
  l1 : F1
  ...
  lm : Fm
} // struct definition
```

$$\frac{\begin{cases} S \to \text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m) \in \Delta \\ \forall i. \Delta A_1, ..., A_n \vdash F_i \ \ \text{type} \end{cases}}{\Delta \vdash \text{struct-decl}(S; A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m) \ \ \text{declaration}}$$

Struct introduction. A qualified struct literal introduces a struct value. It must provide every declared field exactly once. Source field order is not significant; the rule below uses declaration order.

```
S[T1, ..., Tn]::{ l1: e1, ..., lm: em } // qualified struct literal
```

$$\frac{S \to \text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m) \in \Delta \quad \sigma = [T_1/A_1, ..., T_n/A_n]}{\text{fields}(S[T_1, ..., T_n]) = (l_1 : F_1\sigma, ..., l_m : F_m\sigma)}$$

$$\frac{\text{fields}(S[T_1, ..., T_n]) = (l_1 : Q_1, ..., l_m : Q_m) \quad \Gamma \vdash e_1 : Q_1 \quad ... \quad \Gamma \vdash e_m : Q_m}{\Gamma \vdash \text{struct-lit}(S[T_1, ..., T_n], l_1 = e_1, ..., l_m = e_m) : S[T_1, ..., T_n]}$$

Struct field elimination. Field access eliminates a struct value by selecting a declared field type after substituting the struct type arguments.

```
e.l // field access
```

$$\frac{\Gamma \vdash e : S[T_1, ..., T_n] \quad \text{fields}(S[T_1, ..., T_n]) = (..., l : Q, ...)}{\Gamma \vdash e.l : Q}$$

Struct values are also eliminated by struct patterns and may carry contextual forwarding fields; their detailed static rules are specified in “Pattern Matching” and “Structs and Enums”.

3.7 Enum Types

Enum declaration well-formedness. An enum declaration is well-formed when every declared variant payload type is well-formed under the collected type-constructor context and the enum’s type parameters.

```
enum E[A1, ..., An] {
  v1(P11, ..., P1m1)
  ...
  vj(Pj1, ..., Pjmj)
} // enum definition
```

$$\frac{\begin{cases} E \to \mathrm{enum}\big(A_1, ..., A_n; v_1(P_1), ..., v_{q(P_q)}\big) \in \Delta \\ \forall j, k. \Delta A_1, ..., A_n \vdash P_{j,k} \quad \mathrm{type} \end{cases}}{\Delta \vdash \text{enum-decl}\big(E; A_1, ..., A_n; v_1(P_1), ..., v_{q(P_q)}\big) \quad \mathrm{declaration}}$$

Variant constructor introduction. Each declared variant constructor is an introduction form for values of its owning enum type. A variant constructor does not introduce a separate variant type. The payload expressions must match the declared variant payload types after substituting the enum type arguments.

```
E[T1, ..., Tn]::v(e1, ..., em) // qualified variant constructor call
v(e1, ..., em) // unqualified constructor call after unambiguous resolution
```

For an enum declaration shape D, $D \vdash v : (P_1, ..., P_m)$ states that D declares variant v with payload type sequence $(P_1, ..., P_m)$.

The judgment $\Delta\Theta \vdash E[T_1, ..., T_n] :: v \to (Q_1, ..., Q_m)$ instantiates that payload sequence at the enum type arguments.

$$\frac{\begin{cases} E \to D \in \Delta \\ D \vdash v : (P_1, ..., P_m) \\ \mathrm{params}(D) = (A_1, ..., A_n) \\ \sigma = [T_1/A_1, ..., T_n/A_n] \\ \forall i. \Delta\Theta \vdash T_i \quad \mathrm{type} \end{cases}}{\Delta\Theta \vdash E[T_1, ..., T_n] :: v \to (P_1\sigma, ..., P_m\sigma)}$$

$$\frac{\begin{cases} \Delta\Theta \vdash E[T_1, ..., T_n] :: v \to (Q_1, ..., Q_m) \\ \forall k. \Gamma \vdash e_k : Q_k \end{cases}}{\Gamma \vdash \mathrm{variant}(E[T_1, ..., T_n], v, e_1, ..., e_m) : E[T_1, ..., T_n]}$$

An unqualified enum variant expression is typed by the same rule after name resolution has selected exactly one visible enum variant.

Enum match elimination. A match expression eliminates an enum value. For an enum scrutinee type, every declared variant must be covered.

```
match e {
  E::v1(x11, ..., x1m1) => b1
  ...
  E::vq(xq1, ..., xqmq) => bq
}
```

$$\frac{\begin{cases}\Delta\Theta \vdash E[T_1, ..., T_n] :: v \to (Q_1, ..., Q_m) \\ \Gamma, x_1 : Q_1, ..., x_m : Q_m \vdash b : R\end{cases}}{\Gamma \vdash \text{arm}(v(x_1, ..., x_m) \Rightarrow b) : \text{arm-type}(E[T_1, ..., T_n], v, R)}$$

$$\frac{\begin{cases}\Gamma \vdash e : E[T_1, ..., T_n] \\ E \to D \in \Delta \\ D \vdash \text{variants}(v_1, ..., v_q) \\ \forall j. \Gamma \vdash a_j : \text{arm-type}(E[T_1, ..., T_n], v_j, R)\end{cases}}{\Gamma \vdash \text{match}(e; a_1, ..., a_q) : R}$$

Pattern syntax and exhaustiveness diagnostics are specified in “Pattern Matching”.

4 Type Inference

Lane2 permits local type inference only at syntactic positions where the omitted type is determined locally.

A binding’s type is not determined from later uses of the bound name.

Local `let` bindings may omit type annotations only when the initializer has a type without using later references to the bound name.

Function literals may omit parameter types only when the immediately surrounding context provides a function type. Otherwise, every value parameter of a function literal must have an explicit type annotation.

Generic function literals without an immediately surrounding generic function type must explicitly declare type parameters and explicitly annotate value parameters.

Generic function calls and generic data constructors may instantiate type parameters at the use site when the use site is unambiguous.

5 Builtins

`builtin("...")` is an unsafe expression.

The checker does not interpret intrinsic strings. Instead, `builtin` receives its type from direct expected context:

```
fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}
```

This is valid because the function return type provides the expected type `Int` for the body expression.

This is invalid:

```
let x = builtin("%anything")
```

There is no direct expected type.

Incorrect builtin use can produce undefined behavior. Lane2’s type safety guarantee applies only to programs that do not misuse `builtin`.

5.1 Required Intrinsics

A conforming Lane2/Core v1 implementation provides these portable intrinsic names:

Intrinsic	Expected type
`%i64_add`	`(Int, Int) -> Int`
`%i64_sub`	`(Int, Int) -> Int`
`%i64_mul`	`(Int, Int) -> Int`
`%i64_div`	`(Int, Int) -> Int`
`%i64_rem`	`(Int, Int) -> Int`
`%i64_neg`	`(Int) -> Int`
`%i64_equal`	`(Int, Int) -> Bool`
`%i64_less`	`(Int, Int) -> Bool`
`%string_equal`	`(String, String) -> Bool`

Table 2: Required intrinsic names and expected types

Other intrinsic names are implementation-defined unsafe builtins.

`%bool_and`, `%bool_or`, `%bool_not`, and `%bool_equal` are not required intrinsics. The standard prelude defines boolean operations as ordinary Lane2 functions and offered operation values using `if`; `&&` and `||` supply their right operands as thunks.

6 Prelude

The standard prelude is implementation-supplied Lane2 source checked before user code. It is not a module system.

Prelude declarations provide standard operation structs, primitive wrappers around required intrinsics, derived primitive operations written in Lane2, and prelude-provided contextual offers.

Prelude entries are ordinary Lane2 values and types. Operation structs are API conventions that group short operation fields such as `add`, `equal`, and `less`.

Operation laws are API conventions, not compiler-checked rules.

6.1 Standard Prelude Source

The v1 standard prelude source is stored in `lane-std/prelude.lane` and rendered here:

```
// Lane2 v1 standard prelude.
// This file is the source of truth for prelude declarations shown in the
// language specification.

struct Add[T] {
  add : (T, T) -> T
}

struct Sub[T] {
  sub : (T, T) -> T
}

struct Mul[T] {
  mul : (T, T) -> T
}

struct Div[T] {
  div : (T, T) -> T
}

struct Rem[T] {
  rem : (T, T) -> T
}
```

```
struct Neg[T] {
  neg : (T) -> T
}

struct And {
  and : (Bool, () -> Bool) -> Bool
}

struct Or {
  or : (Bool, () -> Bool) -> Bool
}

struct Not {
  not : (Bool) -> Bool
}

struct Equal[T] {
  equal : (T, T) -> Bool
  not_equal : (T, T) -> Bool
}

struct Compare[T] {
  offer equal_impl : Equal[T]
  less : (T, T) -> Bool
  less_eq : (T, T) -> Bool
  greater : (T, T) -> Bool
  greater_eq : (T, T) -> Bool
}

fn[T] op_add(a : T, b : T, auto op : Add[T]) -> T {
  op.add(a, b)
}

fn[T] op_sub(a : T, b : T, auto op : Sub[T]) -> T {
  op.sub(a, b)
}

fn[T] op_mul(a : T, b : T, auto op : Mul[T]) -> T {
  op.mul(a, b)
}

fn[T] op_div(a : T, b : T, auto op : Div[T]) -> T {
  op.div(a, b)
}

fn[T] op_rem(a : T, b : T, auto op : Rem[T]) -> T {
  op.rem(a, b)
}

fn[T] op_neg(value : T, auto op : Neg[T]) -> T {
  op.neg(value)
}

fn op_and(a : Bool, b : () -> Bool, auto op : And) -> Bool {
  op.and(a, b)
}

fn op_or(a : Bool, b : () -> Bool, auto op : Or) -> Bool {
  op.or(a, b)
}
```

```
fn op_not(value : Bool, auto op : Not) -> Bool {
  op.not(value)
}

fn[T] op_equal(a : T, b : T, auto op : Equal[T]) -> Bool {
  op.equal(a, b)
}

fn[T] op_not_equal(a : T, b : T, auto op : Equal[T]) -> Bool {
  op.not_equal(a, b)
}

fn[T] op_less(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.less(a, b)
}

fn[T] op_less_eq(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.less_eq(a, b)
}

fn[T] op_greater(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.greater(a, b)
}

fn[T] op_greater_eq(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.greater_eq(a, b)
}

fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}

fn int_sub(a : Int, b : Int) -> Int {
  builtin("%i64_sub")
}

fn int_mul(a : Int, b : Int) -> Int {
  builtin("%i64_mul")
}

fn int_div(a : Int, b : Int) -> Int {
  builtin("%i64_div")
}

fn int_rem(a : Int, b : Int) -> Int {
  builtin("%i64_rem")
}

fn int_neg(a : Int) -> Int {
  builtin("%i64_neg")
}

fn int_equal(a : Int, b : Int) -> Bool {
  builtin("%i64_equal")
}

fn int_less(a : Int, b : Int) -> Bool {
  builtin("%i64_less")
}

fn string_equal(a : String, b : String) -> Bool {
```

```
  builtin("%string_equal")
}

fn bool_and(a : Bool, b : () -> Bool) -> Bool {
  if a {
    b()
  } else {
    false
  }
}

fn bool_or(a : Bool, b : () -> Bool) -> Bool {
  if a {
    true
  } else {
    b()
  }
}

fn bool_not(a : Bool) -> Bool {
  if a {
    false
  } else {
    true
  }
}

fn bool_equal(a : Bool, b : Bool) -> Bool {
  if a {
    b
  } else {
    bool_not(b)
  }
}

fn[T] make_equal(equal : (T, T) -> Bool) -> Equal[T] {
  Equal::{
    equal,
    not_equal: fn(a : T, b : T) -> Bool {
      if equal(a, b) {
        false
      } else {
        true
      }
    },
  }
}

fn[T] make_compare(
  equal_impl : Equal[T],
  less : (T, T) -> Bool
) -> Compare[T] {
  Compare::{
    equal_impl,
    less,
    less_eq: fn(a : T, b : T) -> Bool {
      if equal_impl.equal(a, b) {
        true
      } else {
        less(a, b)
      }
```

```
    },
    greater: fn(a : T, b : T) -> Bool {
      less(b, a)
    },
    greater_eq: fn(a : T, b : T) -> Bool {
      if equal_impl.equal(a, b) {
        true
      } else {
        less(b, a)
      }
    },
  }
}

let offer int_add_ops : Add[Int] = Add::{ add: int_add }
let offer int_sub_ops : Sub[Int] = Sub::{ sub: int_sub }
let offer int_mul_ops : Mul[Int] = Mul::{ mul: int_mul }
let offer int_div_ops : Div[Int] = Div::{ div: int_div }
let offer int_rem_ops : Rem[Int] = Rem::{ rem: int_rem }
let offer int_neg_ops : Neg[Int] = Neg::{ neg: int_neg }

let int_equal_ops : Equal[Int] = make_equal(int_equal)
let offer int_compare_ops : Compare[Int] = make_compare(int_equal_ops, int_less)

let offer bool_and_ops : And = And::{ and: bool_and }
let offer bool_or_ops : Or = Or::{ or: bool_or }
let offer bool_not_ops : Not = Not::{ not: bool_not }
let offer bool_equal_ops : Equal[Bool] = make_equal(bool_equal)

let offer string_equal_ops : Equal[String] = make_equal(string_equal)
```

Boolean conjunction, disjunction, negation, and equality do not require boolean intrinsics; they can be defined with ordinary `if` expressions in the prelude.

7 Declarations

Declarations introduce types, functions, values, and contextual offers.

7.1 Top-Level Declarations

Top-level declarations are described by the `topLevelDefinition` grammar in “Syntax and Grammar”.

Top-level forms include struct declarations, enum declarations, named function declarations, typed value declarations, offered value definitions, and offer declarations.

7.2 Struct Declarations

Struct declarations use the grammar in “Structs and Enums”.

7.3 Enum Declarations

Enum declarations use the grammar in “Structs and Enums”.

7.4 Function Declarations

Functions are uncurried. There is no automatic currying or partial application.

Function definitions use block bodies:

```
fn add(a : Int, b : Int) -> Int {
  a + b
}
```

There is no `= expr` function body form.

Function result types use `->`, not `:`.

All named functions must state every parameter type and the result type.

Generic named functions place type parameters after `fn` and before the name:

```
fn[A] id(value : A) -> A {
  value
}
```

Function parameters are positional in v1. Labeled function parameters are not supported.

Named function definitions may mark a trailing suffix of parameters with `auto`. These contextual parameters remain ordinary parameters in the function type, but a direct named call may omit them and let Contextual Resolution supply them.

```
fn[T] op_add(a : T, b : T, auto op : Add[T]) -> T {
  op.add(a, b)
}
```

Contextual parameters must appear as a contiguous suffix of the parameter list. Function literals cannot declare `auto` parameters.

A parameter may be marked `offer`; it is then added to the function body's contextual offer environment. The `offer` modifier does not affect the function type or call syntax. Function literals may use `offer` parameters.

```
fn[T] twice(x : T, auto offer add : Add[T]) -> T {
  op_add(x, x)
}
```

Function literals produce function values:

```
let f : (Int, Int) -> Int = fn(a, b) {
  a + b
}
```

Generic function literals place type parameters after `fn`:

```
let id = fn[A](value : A) {
  value
}
```

7.5 Let Declarations

Named top-level `let` declarations must include type annotations.

Local `let` declarations may omit type annotations when their initializer can synthesize a type by local type inference.

An offered value definition has the form `let offer name ... = expression`. It is syntax sugar for a value declaration immediately followed by `offer name`.

Top-level offered value definitions must include type annotations because all top-level value declarations must include type annotations:

```
let offer int_add_ops : Add[Int] = Add::{ add: int_add }
```

Local offered value definitions may omit type annotations when their initializer can synthesize a type:

```
{
  let offer ops = Add::{ add: int_add }
  1 + 2
}
```

Offered value definitions must be named. They are not forward-visible and do not make the defined value available as an offer inside its own initializer.

8 Scopes and Bindings

8.1 Notations

Notation	Meaning
R	resolution environment
Δ	type-name namespace
Γ	ordinary value-binding scope stack
Ω	contextual offer scope stack
s_T, s_V, s_F, s_R	type, value, field, and variant symbols
x	ordinary value name
C	type name
S	struct type constructor
E	enum type constructor
l	struct field name
v	enum variant name
A	contextual parameter target type
$R = (\Delta, \Gamma, \Omega)$	resolution environment components
$\Delta \vdash C \to s_T$	type-name resolution
$\Gamma \vdash x \to s_V$	ordinary value resolution
$\Omega \vdash A \to s_V$	contextual resolution by type
$R \vdash d \to R'$	declaration resolution
$R \vdash e \to e'$	expression-name resolution
$\text{type-of}(s_V) = A$	known type of a value symbol
$\text{forward}(s_V)$	contextual offers forwarded from a struct value
$\text{variant}(s_T, v) = s_R$	variant owned by an enum type symbol

Table 3: Scope and binding notations

8.2 Resolution Model

Name resolution maps source names to symbols before typed core is produced. It does not evaluate expressions. Contextual Resolution is a later type-directed step that supplies omitted contextual arguments from visible offers after ordinary call inference has determined the target contextual parameter types.

The resolution environment has separate namespaces for type names, value names, and contextual offers. Field and variant symbols are owned by nominal type symbols and are not global value bindings.

```
// source names
TypeName
value_name
value_name.field_name
TypeName::variant_name
```

$$R = (\Delta, \Gamma, \Omega)$$

$$\Delta \vdash C \to s_T$$

$$\Gamma \vdash x \to s_V$$

$$\Omega \vdash A \to s_V$$

8.3 Namespaces

Type and value names are resolved by different lookup judgments. The same source spelling may therefore be bound in both namespaces without conflict.

```
enum Option[A] {
  none
  some(A)
}

let Option : Int = 42
```

Type-name lookup. A type name resolves through the type namespace.

$$\frac{C \to s_T \in \Delta}{\Delta \vdash C \to s_T}$$

Ordinary value lookup. A value name resolves through the ordinary value-binding scope stack.

$$\frac{x \to s_V \in \Gamma}{\Gamma \vdash x \to s_V}$$

Ordinary value lookup never searches the contextual offer environment.

Qualified type-member syntax resolves its left-hand side in the type namespace.

```
Option::some(1) // Option is resolved as a type name
```

$$\frac{\Delta \vdash E \to s_T \quad \text{variant}(s_T, v) = s_R}{R \vdash \text{qualified-variant}(E, v) \to s_R}$$

8.4 Top-Level Scope

Top-level `struct`, `enum`, and `fn` declarations are collected before top-level value initializers are resolved. Top-level custom types and functions may therefore refer to each other regardless of textual order.

```
struct S { value : T }
enum T { wrap(S) }
fn f(x : T) -> T { x }
```

$$\frac{\text{types}(d_1, \ldots, d_n) = \Delta \quad \text{functions}(d_1, \ldots, d_n) = \Gamma}{\text{top-recursive}(d_1, \ldots, d_n) = (\Delta, \Gamma)}$$

Top-level value definitions and top-level `offer` declarations are resolved in source order. A top-level `let` initializer may refer only to values and contextual offers available at that declaration point. A top-level function body is resolved after the complete top-level function, value, and contextual offer environments are known.

```
let x : Int = earlier
offer int_add_ops
let offer y_ops : Add[Int] = Add::{ add: int_add }
```

$$\frac{R_i \ ; \ K_i \vdash e_i \to e_{i'} \quad R_{i+1} = R_i, x_i \to s_i}{R_i \vdash \text{top-let}(x_i, T_i, e_i) \to R_{i+1}}$$

$$\frac{\Gamma_i \vdash x \to s_V \quad R_{i+1} = R_i, \text{offer}(s_V)}{R_i \vdash \text{offer-decl}(x) \to R_{i+1}}$$

An offered value definition first binds the value, then offers that value from the declaration point forward.

```
let offer ops : Add[Int] = Add::{ add: int_add }
```

$$\frac{R_i \vdash \text{top-let}(x, T, e) \to R_j \quad R_j \vdash \text{offer-decl}(x) \to R_{j+1}}{R_i \vdash \text{top-let-offer}(x, T, e) \to R_{j+1}}$$

Top-level value initializers are checked only with contextual offers available earlier in source order. Top-level function bodies are checked with the complete top-level contextual offer environment.

8.5 Local Scope

Local `let`, local named `fn`, and local `offer` items are sequential. A local binding is visible only to later local items and the final expression in the same block.

```
{
  let x = e
  fn f(y : T) -> U { b }
  offer ops
  result
}
```

$$\frac{R_i \ ; \ K_i \vdash e_i \to e_{i'} \quad R_{i+1} = R_i, x_i \to s_i}{R_i \vdash \text{local-let}(x_i, e_i) \to R_{i+1}}$$

$$\frac{R_{i+1} = R_i, f \to s_f \quad R_{i+1} \ ; \ K \vdash b \to b'}{R_i \vdash \text{local-fn}(f, U, b) \to R_{i+1}}$$

Local value names may shadow earlier value names from outer layers. Ordinary value bindings in the same layer must have distinct names.

```
let x = outer
{
  let x = inner
  x
}
```

$$\frac{\Gamma = \Gamma_0, (x \to s_1) \quad \Gamma' = \Gamma, (x \to s_2)}{\Gamma' \vdash x \to s_2}$$

8.6 Contextual Offers

`offer x` requires `x` to resolve as a unique ordinary value binding whose type is already known. The operand is a value name, not an arbitrary expression or field path.

```
offer int_add_ops
```

$$\frac{\Gamma \vdash x \to s_V \quad \text{type-of}(s_V) = A}{(\Delta, \Gamma, \Omega) \vdash \text{offer}(x) \to (A \to s_V), \text{forward}(s_V)}$$

A contextual offer affects only Contextual Resolution. It does not introduce an ordinary value binding and does not expose fields as unqualified names.

```
let offer int_add_ops : Add[Int] = Add::{ add: int_add }
1 + 2
```

Multiple visible offers may have the same type. This is not an error at the offer declaration point. Ambiguity is reported only when Contextual Resolution needs a unique value of that type.

```
offer left_add
offer right_add
1 + 2 // ambiguous if both offers have type Add[Int]
```

$$\frac{\Omega \vdash A \to (s_1, ..., s_n) \quad n = 1}{\Omega \vdash \text{contextual}(A) \to s_1}$$

$$\frac{\Omega \vdash A \to ()}{\Omega \vdash \text{contextual}(A) \to \text{error}(\text{missing-offer})}$$

$$\frac{\Omega \vdash A \to (s_1, ..., s_n) \quad n > 1}{\Omega \vdash \text{contextual}(A) \to \text{error}(\text{ambiguous-offer})}$$

Repeating the same offer identity is semantically idempotent but should produce a warning. Contextual offer deduplication uses offer identity, not runtime value equality.

Visible contextual offers from nested lexical scopes are combined rather than shadowed. Nested local function bodies can see contextual offers from their lexical environment.

8.7 Contextual Forwarding Fields

When a value is offered, fields declared with the `offer` modifier are offered recursively. Forwarding contributes only to Contextual Resolution and never to ordinary value lookup.

```
struct Compare[T] {
  offer equal_impl : Equal[T]
  less : (T, T) -> Bool
}

offer int_compare_ops
```

Offering `int_compare_ops : Compare[Int]` also offers `int_compare_ops.equal_impl : Equal[Int]`. Forwarding uses normal nominal field typing of the containing value. Recursive forwarding must detect cycles.

$$\frac{\text{type-of}(s_V) = S[T_1, ..., T_n] \quad \text{offered-fields}(S[T_1, ..., T_n]) = (l_1 : A_1, ..., l_m : A_m)}{\text{forward}(s_V) = (A_1 \to s_V.l_1, ..., A_m \to s_V.l_m), \text{forward}(s_V.l_1), ..., \text{forward}(s_V.l_m)}$$

8.8 Direct Named Calls and Contextual Arguments

A function introduced by a named function definition may declare contextual parameters with `auto`. They must form a contiguous suffix of the parameter list. A contextual parameter remains an ordinary parameter in the function type.

```
fn[T] op_add(a : T, b : T, auto op : Add[T]) -> T {
  op.add(a, b)
}
```

A direct named function call is a call whose callee is a direct value reference to a function definition symbol. Only direct named function calls may omit contextual parameters or use explicit contextual arguments. Calls through ordinary function values must provide every argument positionally according to the function type.

```
op_add(1, 2)
op_add(1, 2, op=custom_add)
```

Explicit contextual arguments are named. Positional call arguments cannot fill contextual parameters.

```
op_add(1, 2, custom_add) // invalid
```

The right-hand side of an explicit contextual argument is an ordinary expression. Explicit contextual arguments participate in ordinary generic call inference before Contextual Resolution supplies the remaining omitted contextual parameters.

Contextual Resolution never infers generic type arguments for the call being completed. It runs only after ordinary positional arguments, explicit contextual arguments, and the direct expected result type have determined the target contextual parameter types.

8.9 Special Resolution Forms

An unqualified enum variant expression may resolve without qualification only when exactly one visible enum variant has that name.

```
some(1)
```

$$\frac{\text{variants-named}(\Delta, v) = (s_R)}{R \ ; \ K \vdash \text{variant-ref}(v) \to s_R}$$

Field access first resolves and checks its base as a unique value, then selects a field owned by the base struct type.

```
ops.op_add
```

$$\frac{R \ ; \ K \vdash x \to s_V \quad \text{type-of}(s_V) = S[T_1, ..., T_n] \quad \text{field}(S, l) = s_F}{R \ ; \ K \vdash \text{field-access}(x, l) \to s_F}$$

Operator aliases first map to fixed ordinary operation names, then elaborate to direct named calls when the operation name resolves to a function definition symbol. `&&` and `||` additionally thunk the right operand before the call is checked.

```
a + b  // elaborates to op_add(a, b)
a && b // elaborates to op_and(a, fn() { b })
```

$$\frac{\text{operator-name}(\texttt{op}) = x \quad \Gamma \vdash x \to s_V}{R \ ; \ K \vdash \text{operator}(\texttt{op}) \to s_V}$$

9 Expressions

Expressions compute values. Lane2 v1 is expression-oriented and has no expression statements.

9.1 Blocks

A block expression contains local items followed by one final expression.

```
{
  let x = 1
  fn double(n : Int) -> Int {
    n + n
  }
  double(x)
}
```

Allowed local items:

- `let`,
- named `fn`,
- `offer`.

Local type definitions are not supported in v1.

A block does not contain expression statements. This is invalid:

```
{
  f(x)
  g(y)
}
```

An empty block is invalid. Write `()` explicitly when a `Unit` value is required.

9.2 Conditional Expressions

`if` is an expression:

```
if condition {
  then_value
} else {
  else_value
}
```

The `else` branch is mandatory.

Both branches must have the same type.

9.3 Calls, Field Access, and Pipeline

Function call:

```
f(x, y)
```

A direct named function call may omit trailing contextual parameters declared with `auto`. The omitted parameters are supplied by Contextual Resolution after ordinary argument checking and generic call inference determine their target types.

```
op_add(1, 2)
```

A caller may explicitly supply contextual parameters by name:

```
op_add(1, 2, op=custom_add)
```

Named call arguments are valid only for contextual parameters of direct named function calls. Non-contextual parameters cannot be supplied by name, and contextual parameters cannot be supplied positionally.

Field access:

```
ops.add
```

The base of field access must resolve and check as a unique value before field selection.

Field access followed by call:

```
ops.add(x, y)
```

This is not method call syntax. Lane2 v1 has no method receiver lookup.

Pipeline syntax is supported:

```
value |> f(a, b)
```

It rewrites to:

```
f(value, a, b)
```

The right-hand side of `|>` must be a call or function literal.

The following are invalid:

```
value |> f
value |> f(_, y)
```

Placeholders are not supported.

10 Structs and Enums

Struct literals are qualified:

```
let p : Point = Point::{ x: 1, y: 2 }
```

Struct field punning is allowed:

```
let p : Point = Point::{ x, y }
```

This is equivalent to:

```
let p : Point = Point::{ x: x, y: y }
```

Struct literals must provide every field exactly once.

The following are not supported:

- anonymous record literals,
- default fields,
- spread,
- copy update,
- field update.

Fields are accessed with dot syntax:

```
p.x
```

V1 has no field visibility modifiers. Every struct field is accessible wherever the struct value is accessible.

10.1 Enums

Enum variants are scoped to their enum.

Variant names may be lowercase or uppercase. Capitalization has no semantic role.

Payloadless variants are values and are written without call parentheses:

```
let x : Option[Int] = Option::none
```

This is invalid:

```
Option::none()
```

Variant payloads are positional:

```
enum Expr {
  int(Int)
  add(Expr, Expr)
}
```

Labeled variant payloads are not part of v1. Named product data must be represented with a struct:

```
struct Node[A] {
  left : Tree[A]
  value : A
  right : Tree[A]
}

enum Tree[A] {
  leaf(A)
  node(Node[A])
}
```

In expressions, unqualified variants are allowed when unambiguous:

```
let x : Option[Int] = some(1)
```

If more than one visible enum has a variant named `some`, the unqualified expression is invalid unless another directly local rule disambiguates it. A qualified variant is always allowed:

```
let x : Option[Int] = Option::some(1)
```

In patterns, variants must always be qualified.

10.2 Contextual Forwarding Fields

A struct field may be marked with `offer`:

```
struct Compare[T] {
  offer equal_impl : Equal[T]
  less : (T, T) -> Bool
  less_eq : (T, T) -> Bool
  greater : (T, T) -> Bool
  greater_eq : (T, T) -> Bool
}
```

When a value of this struct type is offered, contextual forwarding fields are also offered. The forwarded field type is obtained by ordinary nominal field typing of the containing value.

```
offer int_compare_ops
```

If `int_compare_ops : Compare[Int]`, the offer declaration contributes both `Compare[Int]` and `Equal[Int]` offers: the latter is the field path `int_compare_ops.equal_impl`.

Contextual forwarding fields do not expose unqualified field names. They affect only Contextual Resolution. Forwarding is recursive and must detect cycles.

11 Pattern Matching

`match` is an expression:

```
match value {
  Option::none => fallback
  Option::some(x) => x
}
```

Match arms use `pattern => expression`.

Every match must be exhaustive.

V1 patterns:

- wildcard pattern: `_`,
- variable binding pattern: `name`,
- literal pattern,
- qualified enum variant pattern,
- qualified struct pattern.

Enum variants in patterns must be qualified:

```
Option::some(x)
Option::none
```

Bare identifiers in patterns are always variable bindings:

```
match value {
  x => x
}
```

Struct patterns use qualified syntax and must list all fields:

```
match p {
  Point::{ x, y } => x + y
}
```

Struct pattern punning is allowed. Explicit renaming is allowed:

```
match p {
  Point::{ x: a, y: b } => a + b
}
```

Rest patterns, spread patterns, guards, or-patterns, as-patterns, and `is` pattern expressions are not supported in v1.

12 Operators

Operators are aliases for fixed ordinary operation names.

There is no trait, typeclass, or general instance search. An operator is available when the corresponding operation name resolves to a direct named function call whose non-contextual arguments and contextual parameters can be checked.

Primitive operators are not special-cased. For example, `1 + 2` elaborates through the ordinary name `op_add`; the standard prelude defines `op_add` as a generic named function with an `auto` operation parameter.

Recognized operator mappings:

Operator	Operation name
`+`	`op_add`
`-`	`op_sub`
`*`	`op_mul`
`/`	`op_div`
`%`	`op_rem`
unary `-`	`op_neg`
`==`	`op_equal`
`!=`	`op_not_equal`
`<`	`op_less`
`<=`	`op_less_eq`
`>`	`op_greater`
`>=`	`op_greater_eq`
`&&`	`op_and` with a thunked right operand
`||`	`op_or` with a thunked right operand
`!`	`op_not`

Table 4: Recognized operator mappings

Operation names such as `op_add` are ordinary value names, not reserved words. User code may define them subject to ordinary binding uniqueness. Lane2 v1 does not allow user-defined operator tokens or user-defined operator mappings.

`&&` and `||` are short-circuit boolean operators. They elaborate through `op_and` and `op_or`, but the right operand is passed as a zero-argument function instead of being evaluated before the operation call.

`a && b` desugars to a call equivalent to `op_and(a, fn() { b })`. `a || b` follows the same rule with `op_or`. The resulting direct named call may use Contextual Resolution to supply the operation value.

Ordinary calls to `op_and` and `op_or` are strict. Only the `&&` and `||` operator syntaxes thunk the right operand.

Concrete expression precedence, associativity, and unary/binary disambiguation follow the MoonBit parser reference where Lane2 has not deliberately removed a feature.

13 Evaluation

Lane2 v1 is pure and strict.

- Evaluating a safe expression depends only on its inputs and produces a value.
- Function arguments are evaluated before the function body runs.
- `if` and `match` evaluate only the selected branch.
- There is no mutable binding, mutable field, assignment, field update, or implicit statement sequencing.
- IO and other effects are not part of v1 core semantics.

The `Unit` value is written explicitly as `()`.

Empty blocks are invalid. A block must contain exactly one final expression after any local items.

Function calls evaluate all arguments before entering the function body. The only v1 operator forms that delay a subexpression are `&&` and `||`, which pass the right operand as a thunk to the resolved `op_and` or `op_or` value.

14 Undefined Behavior

Incorrect builtin use can produce undefined behavior. Lane2′s type safety guarantee applies only to programs that do not misuse `builtin`.

Invalid integer arithmetic is undefined behavior in v1. This includes signed 64-bit overflow, division by zero, remainder by zero, `MIN_INT / -1`, and negating `MIN_INT`.

15 Conformance

The words “must”, “must not”, “may”, and “should” are normative in this document.

Text marked as rationale, examples, or notes is informative unless it explicitly uses normative language.

An implementation conforms to this draft if it accepts all valid programs described here, rejects invalid programs described here, and gives the specified typing and evaluation behavior for safe programs.

Programs that misuse `builtin` are outside Lane2′s safety guarantee.

16 Appendix

16.1 Deliberately Omitted From V1

V1 omits:

- mutation and assignment,
- field update and copy update,
- modules and imports,
- local type definitions,
- labeled function parameters,
- labeled enum payloads,
- type aliases,
- traits, typeclasses, interfaces,
- method syntax,
- tuple types,
- collections,
- pattern guards,
- or-patterns,
- as-patterns,
- `is` pattern expressions,
- placeholder pipeline.